Fall Guys 스타일 래그돌 아바타 제작 가이드 (Unity 6)

> 이 가이드는 개발 전략 Phase 0 — 래그돌 아바타 제작 단계의 실행 매뉴얼이다. 독립 씬에서 단독으로 완성한 뒤 Phase 0.5 오브젝트 구조에 연결한다.

📋 목차

1. 아바타 메시 & 본 구조 준비
2. Ragdoll Wizard 세팅
3. PhysicsAsset 튜닝
4. RagdollState 상태 머신 구현
5. BlendToAnimation 복귀 구현
6. 코스튬 부속물 SpringJoint
7. IRagdollInput 인터페이스 확정
8. 완료 기준 체크리스트

1️⃣ 아바타 메시 & 본 구조 준비

Fall Guys 스타일 본 구조 요건

Fall Guys 캐릭터는 짧고 뚱뚱한 비율(2등신~2.5등신)이 특징이다. 래그돌이 자연스럽게 작동하려면 본 구조가 Unity Ragdoll Wizard의 요구사항을 충족해야 한다.

```
필수 본 계층 (Ragdoll Wizard 요구사항)
Hips (루트)
├── Spine
│   └── Chest
│       └── Head
├── LeftUpperArm → LeftLowerArm → LeftHand
├── RightUpperArm → RightLowerArm → RightHand
```

```
├── LeftUpperLeg → LeftLowerLeg → LeftFoot
└── RightUpperLeg → RightLowerLeg → RightFoot
```

Unity 6 Import 설정

```
Model Import Settings:
  Animation Type       : Humanoid  ← 반드시 Humanoid
  Optimize Game Object: OFF        ← 래그돌 본 직접 접근 필요
  Skin Weights         : 4 bones   ← 기본값 유지
```

> ⚠️ **Optimize Game Object를 OFF로 해야 하는 이유**: ON이면 본 Transform이 내부로 숨겨져 Ragdoll Wizard가 본을 인식하지 못하고, 런타임에 본 위치 직접 접근이 불가능하다.

2️⃣ Ragdoll Wizard 세팅

Wizard 실행

```
Unity 메뉴 → GameObject → 3D Object → Ragdoll...
```

본 슬롯 매핑

Wizard 슬롯	매핑할 본
Root	Hips
Left / Right Hips	LeftUpperLeg / RightUpperLeg
Left / Right Knee	LeftLowerLeg / RightLowerLeg
Left / Right Foot	LeftFoot / RightFoot
Left / Right Arm	LeftUpperArm / RightUpperArm
Left / Right Elbow	LeftLowerArm / RightLowerArm
Middle Spine	Spine
Head	Head

Wizard 파라미터 (Fall Guys 비율 기준)

```
Total Mass    : 10          ← 가벼운 캐릭터 느낌
Strength      : 50          ← 낮을수록 흐물흐물한 래그돌
Flip Forward  : 확인 필요   ← 캐릭터 전방 방향에 따라 체크
```

> 💡 Strength 값이 높으면 래그돌이 딱딱하게 튕기고, 낮으면 Fall Guys 특유의 흐물흐물한 느낌이 난다. 50을 시작점으로 플레이테스트하며 조정한다.

3️⃣ PhysicsAsset 튜닝

Wizard 실행 후 각 본에 Rigidbody + Collider + CharacterJoint가 자동 생성된다. Fall Guys 스타일에 맞게 수동 튜닝이 필요하다.

Rigidbody 설정 (각 본 공통)

```
Interpolation  : Interpolate        ← 보간으로 떨림 방지
Collision Mode : Continuous Dynamic ← Unity 6 권장
Angular Drag   : 5                  ← 회전 감쇠 (높을수록 덜 뱅글뱅글)
```

Collider 형태 권장

본 부위	권장 Collider	비고
Hips	CapsuleCollider	가장 큰 히트박스
Spine / Chest	CapsuleCollider	세로 방향
Head	SphereCollider	구 형태가 자연스러움
팔 / 다리 상부	CapsuleCollider	Wizard 기본값 유지
팔 / 다리 하부	CapsuleCollider	크기 축소 권장

CharacterJoint 각도 제한 (Fall Guys 비율 기준)

```
[허리 Spine]
  Twist Limit   : -20 ~ 20
  Swing 1 Limit : 30
  Swing 2 Limit : 20
```

```
[어깨 UpperArm]
  Twist Limit   : -60 ~ 60
  Swing 1 Limit : 70
  Swing 2 Limit : 50

[팔꿈치 LowerArm]
  Twist Limit   : -10 ~ 10
  Swing 1 Limit : 90  ← 한쪽 방향만 굽힘
  Swing 2 Limit : 10

[고관절 UpperLeg]
  Twist Limit   : -40 ~ 40
  Swing 1 Limit : 60
  Swing 2 Limit : 40

[무릎 LowerLeg]
  Twist Limit   : -10 ~ 10
  Swing 1 Limit : 80  ← 한쪽 방향만 굽힘
  Swing 2 Limit : 10
```

본 간 충돌 제거 (필수)

래그돌 본끼리 충돌하면 캐릭터가 폭발하듯 튕긴다. 반드시 같은 캐릭터의 본 레이어 간 충돌을 비활성화한다.

```
// 초기화 시 1회 실행
void DisableRagdollSelfCollision()
{
    // 래그돌 전용 레이어 생성: Layer 8 = "RagdollBone"
    Physics.IgnoreLayerCollision(8, 8, true);
}
```

> ⚠️ Project Settings → Physics → Layer Collision Matrix에서 RagdollBone 레이어끼리의 충돌을 OFF로 설정하는 것이 더 안전하다.

4️⃣ RagdollState 상태 머신 구현

전체 구조

```
public class FallGuysRagdoll : MonoBehaviour
{
    public enum RagdollState { Animated, Ragdoll, BlendToAn
im, Dead }

    [Header("References")]
    [SerializeField] private Animator     animator;
    [SerializeField] private Collider     capsuleCollider;
// 이동용 캡슐
    [SerializeField] private Rigidbody    capsuleRb;
// 이동용 Rb

    [Header("Ragdoll Settings")]
    [SerializeField] private float ragdollDuration  = 0.8f;
    [SerializeField] private float blendDuration    = 0.3f;
    [SerializeField] private float impactThreshold  = 5f;

    private RagdollState    currentState = RagdollState.Ani
mated;
    private Rigidbody[]     ragdollRbs;    // 본 Rigidbody
배열
    private Collider[]      ragdollCols;   // 본 Collider
배열
    private Transform[]     ragdollBones;  // 본 Transform
배열

    // BlendToAnim 용 스냅샷
    private Vector3[]    bonePositionSnapshot;
    private Quaternion[] boneRotationSnapshot;
    private float        blendTimer;

    void Awake()
    {
        // Hips 하위 본 전체 수집 (캡슐 제외)
```

```
        ragdollRbs   = GetComponentsInChildren<Rigidbody>
();
        ragdollCols  = GetComponentsInChildren<Collider>();
        ragdollBones = ragdollRbs.Select(rb => rb.transfor
m).ToArray();

        bonePositionSnapshot = new Vector3[ragdollBones.Len
gth];
        boneRotationSnapshot = new Quaternion[ragdollBones.
Length];

        // 초기 상태: Animated
        SetRagdollActive(false);
    }
}
```

SetRagdollActive — 핵심 전환 메서드

```
void SetRagdollActive(bool active)
{
    // 1. 본 Rigidbody 전환
    foreach (var rb in ragdollRbs)
        rb.isKinematic = !active;

    // 2. 본 Collider 전환
    foreach (var col in ragdollCols)
        col.enabled = active;

    // 3. 이동용 캡슐 반전
    capsuleCollider.enabled = !active;
    capsuleRb.isKinematic   = active;

    // 4. Animator 반전 (래그돌 활성 시 OFF)
    animator.enabled = !active;
}
```

OnHitImpact — 충돌 반응

```
public void OnHitImpact(Vector3 impulse, Vector3 hitPoint)
{
    // 임계값 미만이면 무시
    if (impulse.magnitude < impactThreshold) return;
    // 이미 Dead면 무시
    if (currentState == RagdollState.Dead) return;

    currentState = RagdollState.Ragdoll;
    SetRagdollActive(true);

    // 가장 가까운 본에 임펄스 인가
    Rigidbody closestRb = GetClosestBoneRb(hitPoint);
    closestRb.AddForce(impulse, ForceMode.Impulse);
    // 회전 토크 추가 (우스꽝스러운 느낌)
    closestRb.AddTorque(Random.insideUnitSphere * impulse.m
agnitude * 0.5f,
                        ForceMode.Impulse);

    // 0.8초 후 복귀 시작
    StartCoroutine(RagdollToBlendCoroutine());
}

Rigidbody GetClosestBoneRb(Vector3 point)
{
    return ragdollRbs
        .OrderBy(rb => Vector3.Distance(rb.position, poin
t))
        .First();
}

IEnumerator RagdollToBlendCoroutine()
{
    yield return new WaitForSeconds(ragdollDuration);
    if (currentState == RagdollState.Ragdoll)  // Dead로 바
뀌지 않았으면
        StartBlendToAnimation();
}
```

OnDeath — 낙사 처리

```
public void OnDeath()
{
    currentState = RagdollState.Dead;
    SetRagdollActive(true);
    // 복귀 코루틴 중단
    StopAllCoroutines();
    // 입력 차단은 CharacterController에서 Dead 상태 감지로 처리
}
```

5️⃣ BlendToAnimation 복귀 구현

Fall Guys 특유의 "쓰러졌다가 일어나는" 느낌은 BlendToAnimation 단계에서 만들어진다.

StartBlendToAnimation

```
void StartBlendToAnimation()
{
    currentState = RagdollState.BlendToAnim;

    // 1. 현재 래그돌 본 포즈 스냅샷 저장 (반드시 Animator 켜기 전)
    for (int i = 0; i < ragdollBones.Length; i++)
    {
        bonePositionSnapshot[i] = ragdollBones[i].position;
        boneRotationSnapshot[i] = ragdollBones[i].rotation;
    }

    // 2. 본 물리 OFF, 캡슐 물리 ON
    SetRagdollActive(false);

    // 3. 캡슐 위치를 Hips 위치로 이동 (루트 위치 보정)
    Vector3 hipsPos = ragdollBones[0].position;  // [0] = Hips
    capsuleRb.MovePosition(new Vector3(hipsPos.x,
```

```
                                            GetFeetY(),   // 발
높이 기준
                                            hipsPos.z));

    // 4. Animator 활성화 → 기상 애니메이션 재생
    animator.enabled = true;
    animator.CrossFade("GetUp", 0.1f);

    blendTimer = 0f;
}

float GetFeetY()
{
    // 양발 본의 평균 Y좌표를 바닥으로 사용
    float leftFoot  = ragdollBones.First(b => b.name.Contai
ns("LeftFoot")).position.y;
    float rightFoot = ragdollBones.First(b => b.name.Contai
ns("RightFoot")).position.y;
    return Mathf.Min(leftFoot, rightFoot);
}
```

BlendBonesToAnimation — LateUpdate에서 실행

```
void LateUpdate()
{
    if (currentState != RagdollState.BlendToAnim) return;

    blendTimer += Time.deltaTime;
    float t = Mathf.Clamp01(blendTimer / blendDuration);

    // 스냅샷 포즈 → 애니메이션 포즈로 Lerp
    for (int i = 0; i < ragdollBones.Length; i++)
    {
        ragdollBones[i].position = Vector3.Lerp(
            bonePositionSnapshot[i],
            ragdollBones[i].position,  // Animator가 이미 업
데이트한 위치
```

```
            t);
        ragdollBones[i].rotation = Quaternion.Slerp(
            boneRotationSnapshot[i],
            ragdollBones[i].rotation,
            t);
    }

    // 블렌딩 완료
    if (t >= 1f)
    {
        currentState = RagdollState.Animated;
        animator.CrossFade("Locomotion", 0.1f);
    }
}
```

> ⚠️ **Unity 6 주의사항**: LateUpdate에서 bone.position을 덮어쓰면 Animator가 계산한 값을 Override한다. 이것이 BlendToAnim의 핵심 원리다. `animator.enabled = true` 이후 첫 LateUpdate에서만 Animator 결과가 bone에 반영되므로, 스냅샷은 반드시 Animator 켜기 **전**에 저장해야 한다.

6️⃣ 코스튬 부속물 SpringJoint

귀, 꼬리, 안테나 등 부속물은 래그돌 상태와 무관하게 항시 물리가 작동한다. (Fall Guys와 동일한 방식)

컴포넌트 구성

```
[CostumeAccessory]  (귀, 꼬리 등)
  Rigidbody
    - Mass        : 0.05     ← 매우 가볍게
    - Drag        : 2        ← 공기 저항으로 흔들림 감쇠
    - Angular Drag: 5
  SpringJoint
    - Connected Body : 부착 본 (예: Head)
    - Spring         : 80    ← 탄성 강도
    - Damper         : 5     ← 감쇠
    - Min/Max Distance: 0 / 0.05
```

```
  CapsuleCollider
    - IsTrigger : true  ← 물리 충돌 제외, 시각 전용
```

코드 — 본 이동 시 부속물 따라오기

```
public class CostumePhysics : MonoBehaviour
{
    [SerializeField] private Transform  attachBone;   // 부
착 본 (예: Head)
    [SerializeField] private float      springForce = 80f;
    [SerializeField] private float      damperForce = 5f;

    private Rigidbody  rb;
    private SpringJoint joint;

    void Awake()
    {
        rb    = GetComponent<Rigidbody>();
        joint = GetComponent<SpringJoint>();

        joint.connectedBody = attachBone.GetComponent<Rigid
body>();
        joint.spring  = springForce;
        joint.damper  = damperForce;
        joint.minDistance = 0f;
        joint.maxDistance = 0.05f;
    }
}
```

> 💡 부속물 Rigidbody의 `connectedBody`를 부착 본(Head 등)의 Rigidbody로 연결하면, 래그돌 활성/비활성 여부에 관계없이 본이 움직일 때 부속물이 스프링처럼 따라온다.

7️⃣ IRagdollInput 인터페이스 확정

래그돌 시스템을 외부(CharacterController, RopeSystem, PUN2)에서 제어하는 인터페이스. Phase 0에서 실제 구현을 완성한 뒤 이 인터페이스를 확정한다.

```
/// <summary>
/// 래그돌 시스템 외부 제어 인터페이스
/// CharacterController, RopeSystem, PUN2PlayerInput에서 사용
/// </summary>
public interface IRagdollInput
{
    // 현재 상태 조회
    RagdollState CurrentState { get; }
    bool IsRagdollActive => CurrentState is RagdollState.Ragdoll
                                        or RagdollState.BlendToAnim;

    /// <summary>충돌 임펄스 인가. 임계값 미만은 자동 무시.</summary>
    void OnHitImpact(Vector3 impulse, Vector3 hitPoint);

    /// <summary>낙사 처리. Dead 상태로 영구 전환.</summary>
    void OnDeath();

    // 이벤트 (PUN2 RPC 트리거용)
    event Action<Vector3, Vector3> OnImpactTriggered;
    event Action                   OnDeathTriggered;
}
```

연결 지점 요약

호출 주체	호출 메서드	시점
CharacterController	OnHitImpact	장애물 OnCollisionEnter
RopeSystem (별도 담당)	OnHitImpact	고무줄 최대 장력 초과
PUN2PlayerInput	OnHitImpact / OnDeath	RPC 수신 시
ModeManager (별도 담당)	OnDeath	낙사 판정

✅ 완료 기준 체크리스트

Phase 0 완료 기준 (전략 문서와 동일)

- [ ] Ragdoll Wizard 본 매핑 완료, 모든 본에 Rigidbody + Collider + CharacterJoint 생성 확인
- [ ] 본 간 충돌 비활성화 확인 (Physics Layer Matrix)
- [ ] `Animated → Ragdoll` 전환: 임펄스 인가 후 물리 작동 확인
- [ ] `Ragdoll → BlendToAnim` 전환: 0.8초 후 기상 애니메이션 블렌딩 확인
- [ ] BlendToAnim 복귀 시 본이 튀는 현상 없음 (스냅샷 저장 타이밍 검증)
- [ ] `Ragdoll → Dead` 전환: 낙사 후 영구 래그돌 유지 확인
- [ ] 루트 위치 보정: 기상 후 캡슐이 발 위치 기준으로 배치 확인
- [ ] 코스튬 부속물 SpringJoint 동작 확인 (래그돌 활성/비활성 무관)
- [ ] `IRagdollInput` 인터페이스 코드 확정
- [ ] 독립 씬 테스트: CharacterController 없이 임펄스 버튼 하나로 전체 시나리오 재현 가능

다음 단계 진입 조건

모든 체크박스 완료 후 → **Phase 0.5 오브젝트 계층 구조 확정** 진입

`FallGuysRagdoll.cs` 컴포넌트를 MergedBody / AvatarA / AvatarB 각각에 부착